CFGS Desarrollo de Aplicaciones Web · 2º curso

Ejercicios UT3.2

Estructuras de control, arrays y formularios
Arrays en PHP

Desarrollo Web en Entorno Servidor

Alumno: Raúl Sal Romeo

Asignatura: Desarrollo Web en Entorno Servidor (DWES)

Unidad: UT3 — Estructuras de control, arrays y formularios

Actividad: UT3_2 — Ejercicios de arrays

Fecha de entrega: 30 de octubre de 2026

1. Objetivo de la actividad

El objetivo de esta actividad es practicar el manejo de **arrays en PHP**: creación de arrays indexados y asociativos, recorrido con `foreach`, cálculo de valores a partir de sus elementos, ordenación con las funciones nativas del lenguaje y conversión a otros formatos como JSON.

Todos los ejercicios se entregan **dentro de una única página** (`index.php`) que los muestra en pestañas, tal y como pide el enunciado, y todo el HTML generado se ha comprobado con el **validador de W3C**.

Contenido de la entrega

Fichero	Contenido
`index.php`	Página principal con las 8 pestañas; cada pestaña carga un ejercicio.
`ejercicio1.php`	Ejercicio 1 (opcional): array de 10 valores aleatorios entre 1 y 30.
`ejercicio2.php`	Ejercicio 2 (opcional): media, 3 máximos, 3 mínimos, ordenación y borrado de un elemento.
`ejercicio3.php`	Ejercicio 3 (opcional): array multidimensional de países y ciudades.
`ejercicio4.php`	Ejercicio 4 (opcional): ordenación de un array asociativo de las cuatro formas.
`ejercicio5.php`	Ejercicio 5 (opcional): países y capitales ordenados por clave y en mayúsculas.
`ejercicio6.php`	Ejercicio 6 (opcional): conversión del array anterior a JSON con `json_encode`.
`ejercicio7.php`	Ejercicio 7 (**obligatorio**): 50 valores aleatorios y sustitución de los menores que 5 por «suspenso».
`ejercicio8.php`	Ejercicio 8 (**obligatorio**): 10 números aleatorios y suma de pares e impares en una tabla.
`Memoria_UT3_2_SalRomeoRaul.pdf`	Este documento: explicación del trabajo, capturas y validación.
`UT3_2 Ejercicios_Arrays.pdf`	Enunciado de la actividad.

2. Cómo está organizada la aplicación

El enunciado pide que todos los ejercicios se puedan ver en una misma página llamada `index.php` y que estén dentro de pestañas (*tabs*). Para no tener que reescribir la misma estructura HTML nueve veces, he separado el trabajo en dos capas:

- **Un fichero por ejercicio** (`ejercicio1.php` … `ejercicio8.php`): cada uno es una página HTML completa y válida, con su propio `PHP` dentro. Así puedo abrir y probar cada ejercicio por separado.
- **Una página contenedor** (`index.php`) que crea las pestañas de Bootstrap 5 y carga cada ejercicio dentro de un `<iframe>`.

He usado `<iframe>` y no `include` porque cada ejercicio ya trae su propio `<!DOCTYPE>`, `<html>`, `<head>` y `<body>`. Si los incluyera con `include`, el HTML final tendría esas etiquetas repetidas y **no validaría** en W3C. Con el *iframe* cada documento se renderiza de forma independiente y tanto el índice como cada ejercicio son válidos por separado.

Las pestañas del índice

```
<ul class="nav nav-tabs mb-3" id="pestanas" role="tablist">
  <li class="nav-item" role="presentation">
    <button class="nav-link active" id="pestana1" data-bs-toggle="tab"
            data-bs-target="#panel1" type="button" role="tab"
            aria-controls="panel1" aria-selected="true">Ejercicio 1</button>
  </li>
  ...
</ul>

<div class="tab-content" id="contenidoPestanas">
  <div class="tab-pane fade show active" id="panel1" role="tabpanel"
       aria-labelledby="pestana1" tabindex="0">
    <iframe src="ejercicio1.php" title="Ejercicio 1"
            class="w-100 border rounded bg-white" style="height: 560px;"
            loading="lazy"></iframe>
  </div>
  ...
</div>
```

La interactividad de las pestañas la aporta el JavaScript de Bootstrap 5, que se carga desde CDN igual que el CSS. Cada pestaña lleva sus atributos `role`, `aria-controls` y `aria-labelledby` para que también sea accesible con lector de pantalla, y cada *iframe* tiene su `title` descriptivo. Además, cada ejercicio incluye un botón «Volver al índice» para poder navegar también sin depender de las pestañas.



Figura 1. `index.php` con las ocho pestañas de ejercicios.

3. Resolución de los ejercicios

Ejercicio 1 (opcional) — Array de 10 valores aleatorios

Creo el array vacío y lo relleno en un bucle con `random_int(1, 30)`, que devuelve un entero aleatorio criptográficamente seguro dentro del rango. Después lo recorro con `foreach` y muestro cada valor en una lista.

```
$array = [];
for ($i = 0; $i < 10; $i++) {
  $array[] = random_int(1, 30);
}

echo "<ul class='list-group'>";
foreach ($array as $valor) {
  echo "<li class='list-group-item'>$valor</li>";
}
echo "</ul>";
```



Figura 2. Ejercicio 1: array de 10 elementos aleatorios.

Ejercicio 2 (opcional) — Media, máximos, mínimos y ordenación

Es el ejercicio más completo. Sobre el mismo array de 10 valores aleatorios:

1. Calculo la **media** sumando con un `foreach` y dividiendo entre `count($array)`.
2. Para los **3 máximos** copio el array, lo ordeno de mayor a menor con `rsort()` y tomo los tres primeros; para los **3 mínimos** hago lo mismo con `sort()`.
3. Para el **array copiado y ordenado** (`$arrayCopiado`) uso `sort()`.
4. Para **eliminar el segundo elemento** uso `unset($arrayCopiado[1])` (el índice 1 es el segundo) y después `array_values()` para reindexar el array, porque al borrar queda un hueco y el array ya no tendría índices consecutivos.

```
$suma = 0;
foreach ($array as $valor) { $suma += $valor; }
$media = $suma / count($array);

$copiaDesc = $array;  rsort($copiaDesc);   // 3 máximos
$copiaAsc  = $array;  sort($copiaAsc);     // 3 mínimos

$arrayCopiado = $array;
sort($arrayCopiado);

unset($arrayCopiado[1]);
$arrayCopiado = array_values($arrayCopiado);
```

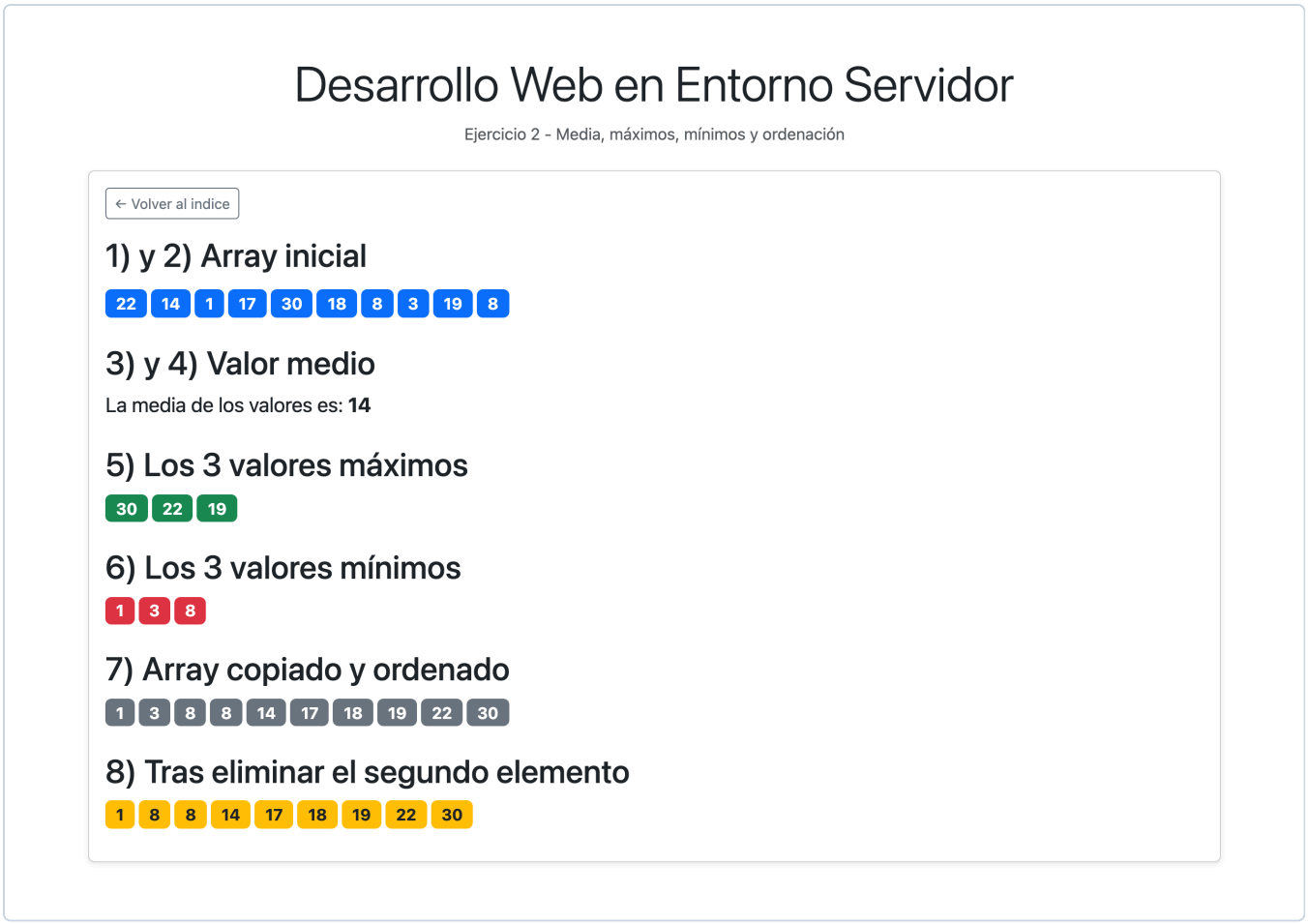


Figura 3. Ejercicio 2: media, máximos, mínimos y ordenaciones.

Ejercicio 3 (opcional) — Array multidimensional de países y ciudades

Uso un array asociativo en el que la **clave es el país** y el **valor es otro array** con tres ciudades. Se recorre con dos `foreach` anidados: el exterior saca el país y el interior sus ciudades. El resultado se pinta en una tabla, una fila por país.

```
$paises = array(
  "España"   => array("Madrid", "Barcelona", "Sevilla"),
  "Francia"  => array("París", "Lyon", "Marsella"),
  "Italia"   => array("Roma", "Milán", "Nápoles"),
  "Alemania" => array("Berlín", "Hamburgo", "Múnich"),
  "Portugal" => array("Lisboa", "Oporto", "Braga")
);

foreach ($paises as $pais => $ciudades) {
  echo "<td><strong>$pais</strong></td>";
  foreach ($ciudades as $ciudad) {
    echo "<span class='badge bg-info text-dark me-1'>$ciudad</span>";
  }
}
```



Figura 4. Ejercicio 3: recorrido de un array multidimensional.

Ejercicio 4 (opcional) — Ordenar un array asociativo de cuatro formas

Parto del array asociativo del enunciado (`Antonio`, `Maria`, `Juan`, `Pepe` con sus notas) y muestro cuatro copias, una por cada función de ordenación:

Función	Qué hace
`asort()`	Ordena de forma ascendente por **valor** manteniendo la relación clave-valor.
`ksort()`	Ordena de forma ascendente por **clave**.
`arsort()`	Ordena de forma descendente por **valor**.
`krsort()`	Ordena de forma descendente por **clave**.

Un detalle importante: estas funciones ordenan **el propio array** (por referencia) y devuelven un booleano, no un array nuevo. Por eso trabajo siempre sobre una copia (`$porValor = $notas;`) y el array original no se modifica entre apartados.

```
$notas = array("Antonio"=>"31", "Maria"=>"28", "Juan"=>"29", "Pepe"=>"27");

$porValor = $notas;  asort($porValor);    // ascendente por valor
$porClave = $notas;  ksort($porClave);    // ascendente por clave
$valorDesc = $notas; arsort($valorDesc);  // descendente por valor
$claveDesc = $notas; krsort($claveDesc);  // descendente por clave
```

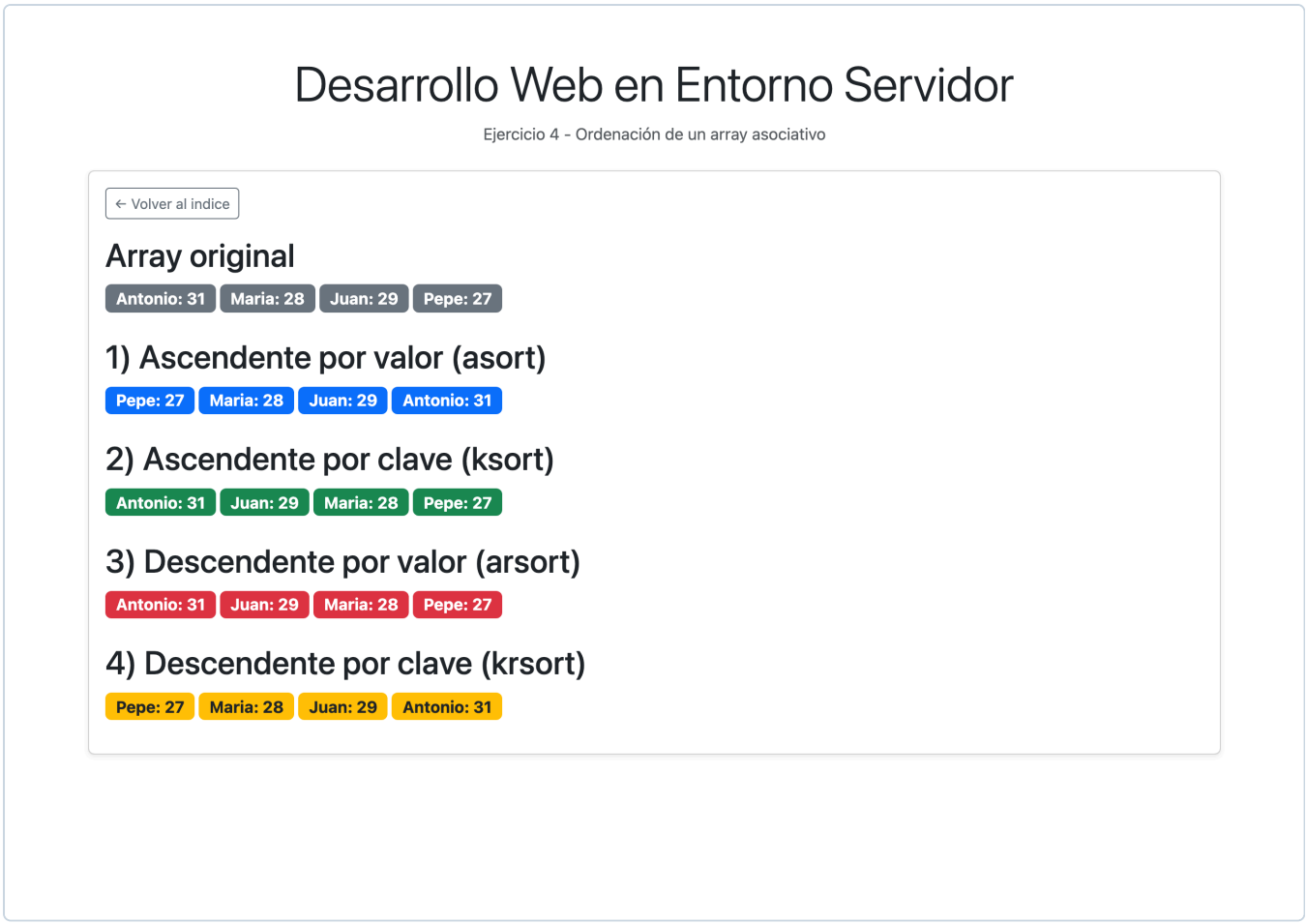


Figura 5. Ejercicio 4: las cuatro ordenaciones del array asociativo.

Ejercicio 5 (opcional) — Países y capitales en mayúsculas y ordenados

Con el array de 25 países y capitales del enunciado: primero lo ordeno por la clave con `ksort()` y después lo recorro mostrando clave y valor en mayúsculas con `strtoupper()`, siguiendo el patrón «*La capital de X es Y*» que se pide.

```
ksort($capitales);

foreach ($capitales as $pais => $capital) {
  echo "<li class='list-group-item'>La capital de "
     . strtoupper($pais) . " es " . strtoupper($capital) . "</li>";
}
```

Nota: `strtoupper()` solo transforma correctamente los caracteres ASCII, que es lo que necesitan los nombres del enunciado (*ITALY*, *ROME*…). Si los nombres llevaran acentos habría que usar `mb_strtoupper()` de la extensión `mbstring`.



Figura 6. Ejercicio 5: capitales en mayúsculas ordenadas por país.

Ejercicio 6 (opcional) — Convertir el array a JSON

Reutilizo el array del ejercicio anterior y lo convierto con `json_encode()`. Como un array asociativo en PHP se codifica como objeto JSON, el resultado son pares `"PAIS": "CAPITAL"`. Uso dos opciones:

- `JSON_PRETTY_PRINT`: añade saltos de línea y sangría para que se lea bien.
- `JSON_UNESCAPED_UNICODE`: evita que los caracteres no ASCII se guarden como `\uXXXX`.

Al imprimir el JSON dentro de `<pre>` lo paso por `htmlspecialchars()`, porque el JSON puede contener comillas y ángulos que romperían el HTML. Es la forma correcta de volcar texto generado por el programa dentro de una página.

```php
$json = json_encode($capitales, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);

echo "<pre class='bg-light border rounded p-3'>"
   . htmlspecialchars($json) . "</pre>";
```



Figura 7. Ejercicio 6: resultado de `json_encode()`.

Ejercicio 7 (obligatorio) — 50 valores y sustitución por «suspenso»

Genero 50 valores aleatorios entre 1 y 10 con `random_int()` y los muestro. Después recorro el array **por índice** (`for`, no `foreach`) porque necesito modificar cada posición: si el valor es menor que 5 lo cambio por la cadena `"suspenso"`. Al mostrarlo por segunda vez coloreo en rojo los suspensos y en verde el resto, para que se vea claramente el cambio.

```php
$array = [];
for ($i = 0; $i < 50; $i++) {
  $array[] = random_int(1, 10);
}

for ($i = 0; $i < 50; $i++) {
  if ($array[$i] < 5) {
    $array[$i] = "suspenso";
  }
}
```

Detalle: al comparar después uso `=== "suspenso"` (comparación estricta) para distinguir la cadena de un número, y el array pasa a ser **mixto** (enteros y cadenas), algo que PHP permite sin problemas porque es un lenguaje de tipado dinámico.



Figura 8. Ejercicio 7: array original y array con los valores menores que 5 sustituidos.

Ejercicio 8 (obligatorio) — Suma de pares e impares en tabla

Genero 10 enteros aleatorios entre 1 y 100, los muestro y los recorro una sola vez: si el resto de dividir entre 2 es 0 sumo a `$sumaPares` y si no, a `$sumaImpares`. El resultado se presenta en una tabla de Bootstrap con dos filas. Tal y como pide el enunciado, **solo** se usan arrays y estructuras de control (bucle + `if/else`).

```
$sumaPares = 0;
$sumaImpares = 0;
foreach ($numeros as $valor) {
  if ($valor % 2 == 0) {
    $sumaPares += $valor;
  } else {
    $sumaImpares += $valor;
  }
}
```



Figura 9. Ejercicio 8: números generados y tabla con la suma de pares e impares.

4. Validación del HTML en W3C

El enunciado pide comprobar que el HTML valida en el validador de W3C. En este proyecto hay que tener en cuenta un detalle: **no se puede validar el código fuente** `.php`, porque el validador necesita el HTML ya generado; si le pasamos el fichero tal cual, las etiquetas `<?php ?>` no son HTML.

El proceso que he seguido es:

1. Ejecutar el proyecto en local con el servidor de PHP (XAMPP).
2. Pedir al navegador el **HTML renderizado** de cada página (`index.php` y `ejercicio1.php` … `ejercicio8.php`).
3. Enviar ese HTML al **validador Nu de W3C** (`validator.w3.org/nu/`) usando la pestaña *text input*, que es la misma vía que la web oficial con «Check by direct input».

4. Revisar que no haya errores ni avisos.

Resultado de la validación

Página validada	Errores	Resultado W3C
`index.php`	0	**Válido, sin avisos**
`ejercicio1.php`	0	**Válido, sin avisos**
`ejercicio2.php`	0	**Válido, sin avisos**
`ejercicio3.php`	0	**Válido, sin avisos**
`ejercicio4.php`	0	**Válido, sin avisos**
`ejercicio5.php`	0	**Válido, sin avisos**
`ejercicio6.php`	0	**Válido, sin avisos**
`ejercicio7.php`	0	**Válido, sin avisos**
`ejercicio8.php`	0	**Válido, sin avisos**

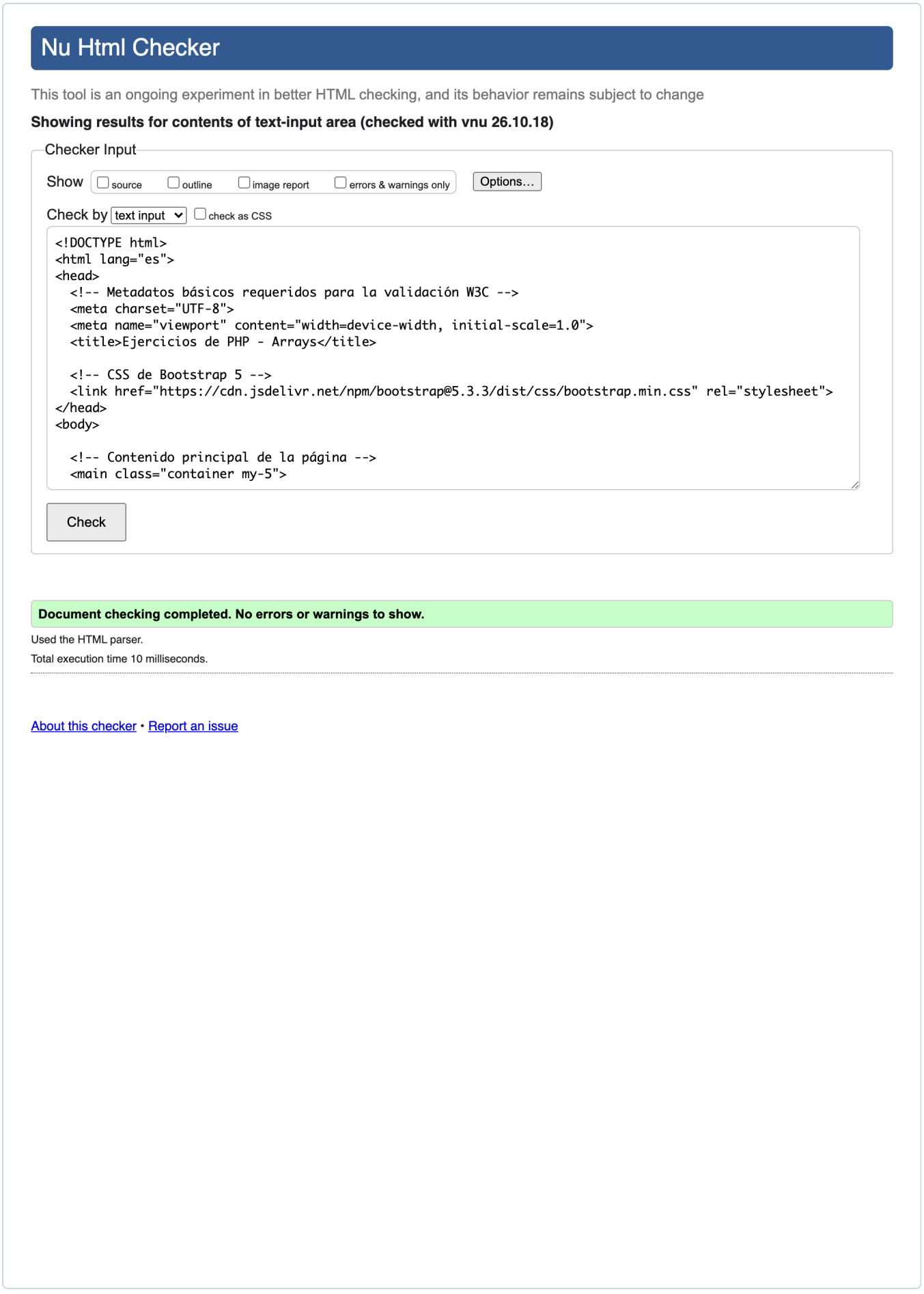


Figura 10. W3C: `index.php` «Document checking completed. No errors or warnings to show.»

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for contents of text-input area (checked with vnu 26.10.18)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options…

Check by text input ☐ check as CSS

```
<!DOCTYPE html>
<html lang="es">
<head>
  <!-- Metadatos básicos requeridos para la validación W3C -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio 2 - Operaciones con arrays</title>

  <!-- CSS de Bootstrap 5 -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

  <!-- Contenido principal de la página -->
  <main class="container my-5">
```

Check

Document checking completed. No errors or warnings to show.

Used the HTML parser.

Total execution time 3 milliseconds.

About this checker • Report an issue

Figura 11. W3C: `ejercicio2.php` sin errores ni avisos.

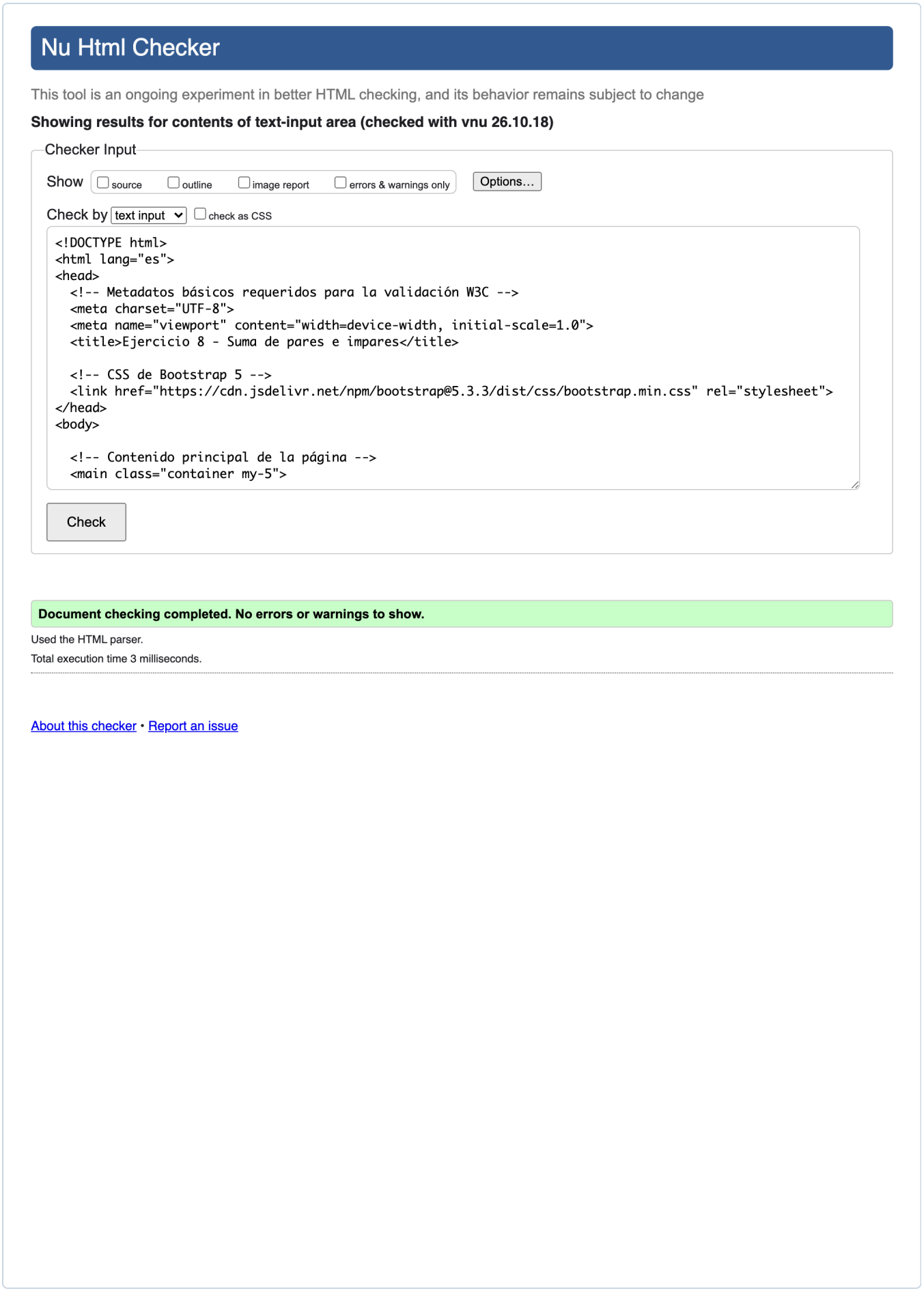


Figura 12. W3C: `ejercicio8.php` sin errores ni avisos.

5. Qué he aprendido

- **Arrays indexados y asociativos** se crean y se recorren igual con `foreach`, pero en los asociativos conviene recorrer `$clave => $valor` para no perder el nombre del estudiante, del país, etc.
- Las funciones de ordenación (`sort`, `rsort`, `asort`, `ksort`, `arsort`, `krsort`) **modifican el array original** y devuelven un booleano: si quiero conservar el original tengo que trabajar sobre una copia.
- Al borrar un elemento con `unset()` el array **no se reindexa**; para eso está `array_values()`. Los índices que no son consecutivos dan problemas al recorrer con `for`.
- Para **modificar** los valores de un array hace falta el índice (`for`); el `foreach` por valor trabaja sobre una copia del elemento, no sobre el array original.
- `json_encode()` convierte un array asociativo en un objeto JSON, y las opciones `JSON_PRETTY_PRINT` y `JSON_UNESCAPED_UNICODE` hacen el resultado legible.
- Cuando genero HTML desde PHP debo escapar el contenido con `htmlspecialchars()` para que comillas o ángulos no rompan la página.
- En el HTML, un *iframe* permite tener varios documentos completos en una misma página, algo útil para este tipo de entregas. Si usara `include` se duplicarían `html`, `head` y `body` y el documento dejaría de ser válido.
- Validar en W3C tiene truco: **hay que validar el HTML renderizado**, no el código PHP.

6. Cómo ejecutar el proyecto

Con XAMPP, basta con copiar la carpeta del proyecto dentro de `htdocs` y abrir en el navegador:

```
http://localhost/UT3_2 - Ejercicios Arrays Archivo/index.php
```

También funciona con el servidor embebido de PHP, sin necesidad de Apache:

```
cd "UT3_2 - Ejercicios Arrays Archivo"
php -S localhost:8000
# y abrir http://localhost:8000/index.php
```

No hay base de datos ni configuración adicional: los datos de todos los ejercicios se generan en el propio script en cada ejecución, por lo que los valores aleatorios cambian cada vez que se recarga la página.